Name: Joseph Nishimwe
Email: j.nishimwe@alustudent.com
Github : github.com/josep-prog
Date: 21st June 2026
DemoVideo: https://youtu.be/bKbpwR9TH1k

Name	Link/destination
SummativeProject_DSA	SummativeProject_DSA
Question 1: Emergency Dispatch Incident Tracker	Emergency Dispatch Incident Tracker
Question 2: Secure Maintenance Procedure Validator	Question 2: Secure Maintenance Procedure Validator
Question 3: Airline Route Relationship Analyzer	Question 3: Airline Route Relationship Analyzer
Question 4: Campus Delivery Robot Navigation	Question 4: Campus Delivery Robot Navigation
Question 5 : Telemetry Data Compression Utility	Question 5 : Telemetry Data Compression Utility

Summative Project Report

Abstract

This report documents the design, implementation, and testing of five independent C programs, each solving a distinct real-world problem using a data structure and algorithm chosen to fit that problem's constraints rather than a single reused approach: a doubly linked list for chronological incident tracking, a balanced Binary Search Tree (BST) for procedure validation, a directed graph with an adjacency matrix for route analysis, a weighted graph with Dijkstra's algorithm for shortest-path navigation, and Huffman coding for lossless telemetry compression. For each system, the report covers the problem requirements, the rationale behind the chosen data structure and algorithm, implementation details with time/space complexity, the testing methodology and observed results, and the challenges encountered during development.

Compilation and Execution

All five programs are written in standard C and were compiled with GCC on Linux. Each program is self-contained in a single .c file and is compiled and run as follows:

Question			
1	gcc incidentTracker.c -o incidentTracker	./incidentTracker	None (generates incident_log.txt on quit)
2	gcc validator.c -o validator	./validator	procedures.txt (in the same directory)
3	gcc analyzer.c -o analyzer	./analyzer	None
4	gcc navigation.c -o navigation	./navigation	None
5	gcc utility.c -o utility	./utility	telemetry.txt (in the same directory)

Question 1: Emergency Dispatch Incident Tracker

Data Structure and Algorithm Justification

To solve this problem, I used a **doubly linked list** because the system requires moving both forward and backward through incidents. This structure allows efficient navigation in both directions, which fits the operator commands (f and b) naturally.

My approach was to always insert new incidents at the end of the list to maintain chronological order. I also maintained three pointers: *oldest*, *newes*t, and *current*. This made it easier to manage navigation and live monitoring at the same time. This approach is important because the system must support **continuous incoming data while still allowing users to browse history**, which is not easy to achieve with simpler structures like arrays, since arrays would require shifting elements whenever the oldest entry is discarded.

Implementation Details and Complexity

Each Incident node stores an *id, source, description,* a generated *timestamp,* and *older/newer* pointers. *addIncident()* appends in O(1) time by updating *newest->newer* and the new node's older pointer; it never has to traverse the list. *removeOldest()* is also O(1), unlinking the head and reassigning current if it pointed at the deleted node, so navigation never dereferences a freed pointer.

goForward()/goBack() are O(1) per step since they simply follow a single pointer. *clearAll()* and *saveSession()* are O(n) because they must visit every node once to free memory or write it to incident_log.txt. Capacity enforcement *(MAX_INCIDENTS = 25)* is handled by calling *removeOldest()* before inserting, which keeps the list size bounded by a single O(1) check on every insertion rather than a periodic O(n) cleanup.

Testing Methodology and Results

The program was compiled with *gcc incidentTracker.c -o incidentTracker* and exercised through its command interface *(v, f, b, l, s, d, q*):

- **Boundary navigation**: with only one incident in the list, pressing f returned "Already at newest incident." and b returned "Already at oldest incident.", confirming the boundary checks work instead of dereferencing a NULL pointer.

- **Live monitoring + browsing concurrency**: enabling l injected a new incident automatically each loop cycle, while a subsequent v still showed the original current incident (ID 1) rather than jumping to the newest one confirming that live insertion does not disturb the operator's browsing position, exactly as the requirement specifies.

- **Capacity enforcement**: incidents were generated past the 25-incident cap; each insertion beyond the cap triggered removeOldest() first, keeping totalIncidents at 25 and the oldest record correctly discarded.

- **Delete and save**: d cleared the list ("All incidents deleted.") and q wrote the remaining session to incident_log.txt before exiting cleanly, confirmed by inspecting the file contents after the run.

Challenges and Solutions

One challenge I faced was handling the **maximum capacity of 25 incidents**. Initially, I added new incidents before removing old ones, which caused inconsistencies. I fixed this by removing the oldest incident before inserting a new one. Another issue was incorrect pointer updates after deletion, which broke navigation. I solved this by carefully updating all related pointers (including reassigning *current* and *newest*) immediately after removing a node.

Question 2: Secure Maintenance Procedure Validator

Data Structure and Algorithm Justification

For this system, I used a **Binary Search Tree (BST)** to store and search maintenance procedures. The main reason for this choice was to achieve **fast lookup performance**, which is required when validating technician input.

Rather than inserting procedure names in file order (which can produce a degenerate, list-like tree if the file happens to be alphabetically sorted), I sort and de-duplicate the names first and build the tree with *buildBalanced(),* which always picks the middle element as the root recursively. This guarantees a height-balanced tree and O(log n) lookup regardless of the order procedures appear in the file.

Implementation Details and Complexity

search() runs in O(log n) thanks to the balanced construction. When no exact match exists, *findClosest()* computes a Levenshtein edit distance *(similarity(),* an O(n·m) dynamic-programming routine comparing the input against every stored name) for every node in the tree and keeps the lowest-scoring candidate, an O(N·n·m) operation overall where N is the number of procedures (bounded by 50) acceptable given the small, fixed dataset size specified by the requirements. A suggestion is only returned if its edit distance is no more than half the input's length, which filters out unrelated names instead of always suggesting something. Unknown commands are rejected and appended to *audit.log* via *logAttempt().* All nodes are freed recursively with *freeTree()* on exit, and a fixed-width *scanf("%49s", input)* is used to prevent a buffer overflow on the 50-byte input array.

Testing Methodology and Results

Compiled with *gcc validator.c -o validator* and run against the 10 sample procedures in *procedures.txt (LOCK_PANEL, RESET_SENSOR, CALIBRATE_ARM, RESTART_LINE, INSPECT_VALVE, REPLACE_FILTER, TIGHTEN_BOLT, CHECK_PRESSURE, LUBRICATE_GEAR, SHUTDOWN_MOTOR):*

Input	Expected Behavior	Actual output
LOCK_PANEL	Exact match approved	APPROVED: LOCK_PANEL
LOC_PANEL	Close match suggested	UNKNOWN COMMAND. Did you mean: LOCK_PANEL ?
FLY_TO_MOON	No close match rejected and logged	REJECTED: FLY_TO_MOON and a TIME / COMMAND: FLY_TO_MOON / STATUS: REJECTED entry written to audit.log
"Show All Procedures" (option 2)	In-order traversal prints names alphabetically	CALIBRATE_ARM, CHECK_PRESSURE, INSPECT_VALVE, LOCK_PANEL, LUBRICATE_GEAR, REPLACE_FILTER, RESET_SENSOR, RESTART_LINE, SHUTDOWN_MOTOR, TIGHTEN_BOLT

All three verification paths (exact, similar, unknown) behaved as specified, and audit.log correctly recorded only the rejected attempt.

Challenges and Solutions

One challenge I encountered was implementing a reliable similarity check. Early versions suggested incorrect procedures. I solved this by introducing a threshold (half the input length)

to ensure only meaningful matches are suggested instead of always returning the "closest" name even when it is unrelated. Another challenge was memory management: since nodes are dynamically allocated, I implemented a recursive *freeTree()* function to free all nodes after execution and confirmed no procedure is leaked by tracing every *malloc()* in *createNode()* to a corresponding *free()*.

Question 3: Airline Route Relationship Analyzer

This problem was solved using a **directed graph**, where airports are represented as nodes and flight routes as edges. I used an **adjacency matrix** to store the connections. I chose this approach because it allows quick checking of whether a route exists between two airports, and it also makes it easy to generate the required matrix representation. The system supports both outgoing and incoming route queries. It also allows dynamic updates such as adding or removing airports and routes.

This approach is important because airline routes are naturally directional, and the system needs to clearly distinguish between incoming and outgoing flights.something an undirected structure could not represent correctly.

Implementation Details and Complexity

findAirport() performs a linear scan, O(n) in the number of airports (bounded at 20). *addRoute()/removeRoute()* are O(n) for the lookup plus O(1) for the matrix write. *showOutgoing()/showIncoming()* scan one row or column of the matrix, O(n). *removeAirport()* is the most expensive operation at $O(n^2)$: it must shift every row and column past the removed index to close the gap in the matrix, then shift the airport name array. *showMatrix()* is $O(n^2)$ since it must print every cell.

Testing Methodology and Results

Compiled with gcc analyzer.c -o analyzer and tested against the seven sample airports and six routes from the assignment (KGL →NBO, KGL → EBB, NBO→JNB, EBB→ADD, ADD→CAI, JNB→CPT):

Test	Action	Result
Outgoing query	KGL outgoing	-> NBO, -> EBB (correct: matches both KGL edges)
Incoming query	JNB incoming	<- NBO (correct: only NBO routes into JNB)
Matrix accuracy	Display matrix	7×7 matrix with a 1 exactly at the six edges listed above, 0 everywhere else
Removal consistency (no dependency)	Remove JNB, re-check KGL outgoing	Unchanged: -> NBO, -> EBB (JNB removal does not affect unrelated rows)
Removal consistency (with dependency)	Remove EBB, re-check KGL outgoing	Correctly drops to -> NBO only, and the regenerated 6×6 matrix shows EBB's row/column fully removed with all remaining routes intact

These results confirm that the matrix is correctly re-indexed after a deletion rather than left with stale or shifted entries.

Challenges and Solutions

One challenge I faced was maintaining consistency when removing airports. Initially, some connections were left incorrect after deletion. I fixed this by updating all related rows and columns in the matrix through an explicit shift routine. Another challenge was ensuring the adjacency matrix always reflects the current state after updates. I solved this by re-deriving and re-displaying the matrix from the live *graph[][]* array after every modification rather than caching a separate copy.

Question 4: Campus Delivery Robot Navigation

To solve this problem, I used a **weighted graph** where each building is a node and each path is an edge with a distance. To find the shortest route, I implemented **Dijkstra's Algorithm**. I chose this approach because the problem requires finding the **minimum distance path**, and Dijkstra's algorithm is specifically designed for this type of situation. It efficiently calculates the shortest path from a starting point to a destination. My approach was to represent the

campus using an adjacency matrix and then apply Dijkstra's algorithm to compute the shortest path to the Dormitory. I also stored parent nodes to reconstruct and display the actual path taken.

Implementation Details and Complexity

With 7 buildings, the graph is represented as a 20×20 adjacency matrix (*graph[i][j]*), and *addEdge()* writes both *graph[i][j*] and *graph[j][i]* since campus paths are undirected. *dijkstra()* uses the classic array-based formulation: at each of the n−1 iterations it scans all n nodes to find the unvisited node with the smallest tentative distance (O(n)), giving an overall $O(n^2)$ running time. This is the appropriate choice here rather than a heap-based O((V+E) log V) implementation, since the graph has only 7 nodes and a priority queue would add complexity with no measurable benefit. The *parent[]* array lets the path be reconstructed in O(path length) by walking backwards from the target to the start.

Testing Methodology and Results

Compiled with *gcc navigation.c -o* navigation and tested against the 7-building campus graph from the assignment:

Start building	Computed shortest path	Total distance
Library	Library → Cafeteria → Charging Station → Administration → Dormitory	15
Charging Station	Charging Station → Administration → Dormitory	7
Invalid name ("Gym")	—	Invalid building name. (handled gracefully, no crash)

The Library result is notable: the direct-looking route via Science Block (Library → Cafeteria → Science Block → Dormitory = 6+4+8 = 18) and the route via Engineering (Library → Engineering → Administration → Dormitory = 15+5+3 = 23) are both longer than the path the algorithm actually found through the Charging Station (6+2+4+3 = 15), confirming

Dijkstra is correctly comparing all candidate paths rather than greedily following the most "obvious" route.

Challenges and Solutions

One challenge I faced was handling invalid building names entered by users. Initially, the system would fail or behave unexpectedly. I solved this by validating input with *find()* before running the algorithm, returning a graceful error instead of an undefined array index. Another challenge was correctly updating distances during algorithm execution. I fixed this by carefully following the standard Dijkstra relaxation step *(dist[u] + graph[u][v] < dist[v])* and verifying the result against multiple manually-computed paths, as shown in the table above.

Question 5: Telemetry Data Compression Utility

For this system, I used **Huffman Coding**, a lossless compression algorithm that reduces file size without losing any information. I chose this approach because the problem requires efficient data compression while ensuring the original data can be fully recovered. Huffman coding achieves this by assigning shorter codes to frequently occurring characters and longer codes to less frequent ones.

My approach involved reading the telemetry file, calculating character frequencies, building a Huffman tree, and then encoding the data into a compressed file. During decompression, the tree is reconstructed to restore the original file. This approach is important because it reduces the amount of data transmitted over limited bandwidth systems like satellites, while still preserving data integrity. building an optimal prefix-code binary tree from character frequencies.

Implementation Details and Complexity

countFreq() scans the input file once, O(file size), to build a 256-entry frequency table. *buildTree()* repeatedly removes the two lowest-frequency nodes (*removeMin()*, a linear scan, O(k) per call) and merges them, for $O(k^2)$ total where $k \leq 256$ is the number of distinct byte values bounded and fast regardless of file size. *makeCode()* walks the resulting tree once, O(k), assigning a bit-string to each leaf. Encoding the file is O(file size × average code length).

Rather than storing a fixed 256-entry frequency table in the compressed file's header (which would waste space on small files), the format stores only the symbols actually present (*numSymbols*, then each *symbol + freq* pair) followed by the total bit count, keeping header overhead proportional to the alphabet actually used. Decompression rebuilds an identical tree deterministically from the stored frequencies using the same *buildTree()* function, then walks the tree bit-by-bit (*O(totalBits)*) to recover each original byte. A single-character edge case (a file with only one distinct byte, which would otherwise produce a single-leaf tree with no left/right code) is special-cased to emit one bit per occurrence.

Testing Methodology and Results

Compiled with *gcc utility.c -o utility* and run against a 712-byte *telemetry.txt* sample log (timestamps, temperature, humidity, battery, and alert-status readings, including a *WARNING* and a *CRITICAL* entry to vary the character distribution):

REPORT

Original Size : 712 bytes

Compressed Size : 625 bytes

Compression Ratio : 12.22%

Verification : SUCCESS

verify() performs a byte-by-byte comparison between telemetry.txt and the decompressed *telemetry_restored.txt* and confirmed the files are identical, proving the compression is fully lossless. The compression ratio is modest (12.22%) because the sample log is short and its character set is fairly evenly distributed across digits and letters Huffman's advantage grows with file size and skew in character frequency, which was confirmed by re-running the test on a version of the file with the six log entries duplicated several times: the compression ratio improved as repeated structure (timestamps, field labels) gave high-frequency characters shorter codes.

Challenges and Solutions

One challenge I encountered was reconstructing the Huffman tree correctly during decompression. Early attempts produced incorrect outputs. I solved this by storing enough information in the compressed file to rebuild the tree accurately. Another challenge was

verifying correctness. I implemented a comparison step to ensure the decompressed file is exactly the same as the original.

Testing

Question	Data structure / Algorithm	Core Complexity	Verified Behaviour
1	Doubly linked list	O(1) insert/navigate, O(n) clear/save	Boundary navigation, capacity cap at 25, live-monitoring concurrency, session save
2	Balanced BST + edit distance	O(log n) search, O(N·n·m) suggestion	Exact match, similar-match suggestion, rejection + audit logging
3	Directed graph (adjacency matrix)	O(n) query, O(n^2) removal/display	Outgoing/incoming queries, matrix accuracy, removal re-indexing
4	Weighted graph + Dijkstra	O(n^2) shortest path	Shortest path correctness against manually computed alternatives, invalid-input handling
5	Huffman coding	O(k^2) tree build, O(file size) encode/decode	Compression ratio, byte-for-byte lossless restoration

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3rd ed.). MIT Press.

GeeksforGeeks. (n.d.). Doubly linked list. Retrieved June 21, 2026, from https://www.geeksforgeeks.org/doubly-linked-list/

GeeksforGeeks. (n.d.). Binary search tree data structure. Retrieved June 21, 2026, from https://www.geeksforgeeks.org/binary-search-tree-data-structure/

GeeksforGeeks. (n.d.). Graph and its representations. Retrieved June 21, 2026, from https://www.geeksforgeeks.org/graph-and-its-representations/

GeeksforGeeks. (n.d.). Dijkstra's shortest path algorithm. Retrieved June 21, 2026, from https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/

GeeksforGeeks. (n.d.). Huffman coding | Greedy algorithm. Retrieved June 21, 2026, from https://www.geeksforgeeks.org/huffman-coding-greedy-algo-3/

cppreference.com. (n.d.). C standard library. Retrieved June 21, 2026, from https://en.cppreference.com/w/c